

LABORATORY 3

DESIGN OF COMBINATIONAL AND SEQUENTIAL CIRCUITS

OBJECTIVES

No.	Topics	Requirements
1	Design a combinational circuit: ➤ A 1-bit Full Adder.	▪ Complete the schematic and symbol for each component, then run a transient simulation. ▪ Define the speed of your design. ▪ Create layouts for each logic gate, then show DRC confirmation and corresponding schematic with proof of LVS.
2	Design a sequential circuit: ➤ A PRBS generator.	▪ Complete the schematic and symbol for each component, then run a transient simulation. ▪ Define the clock frequency and maximum frequency of your design.
3	Design a VerilogA model: ➤ 4-bit counter-up. ➤ 4-bit one-hot bubble code decoder. ➤ Implement a testbench with a 16-to-1 MUX.	▪ Complete the VerilogA code for each block with a compilable symbol. ▪ Complete the testbench for each block. ▪ Implement a full testbench with both schematic and VerilogA view (MUX 16-1 testbench).

EXPERIMENT 1

Objective: Design a combinational circuit – a 1-bit Full Adder.

Requirements:

- Complete the truth table, schematic, and symbol for each component, then run a transient simulation.
- Define the speed of your design.
- Create layouts for each logic gate, then show DRC confirmation and corresponding schematic with proof of LVS.

Instructions:

➤ In this section, students review a 1-bit Full Adder (FA). Moreover, FA will be observed in another aspect – topology. First, let's review the truth table of this component.

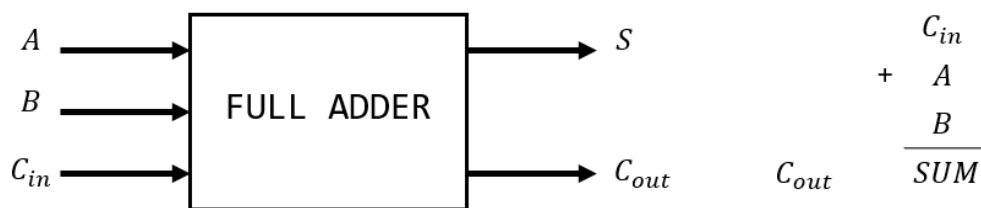


Figure 1 Block diagram of FA.

A	B	C _{in}	S	C _{out}	Carry Status
0	0	0	0	0	Delete
0	0	1	1	0	Delete
0	1	0	1	0	Propagate
0	1	1	0	1	Propagate
1	0	0	1	0	Propagate
1	0	1	0	1	Propagate
1	1	0	0	1	Generate
1	1	1	1	1	Generate

Table 1 The truth table of a 1-bit Full Adder.

$$S = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in} = A \oplus B \oplus C_{in}$$

$$C_{out} = BC_{in} + AC_{in} + AB + ABC_{in} = (A + B)C_{in} + AB$$

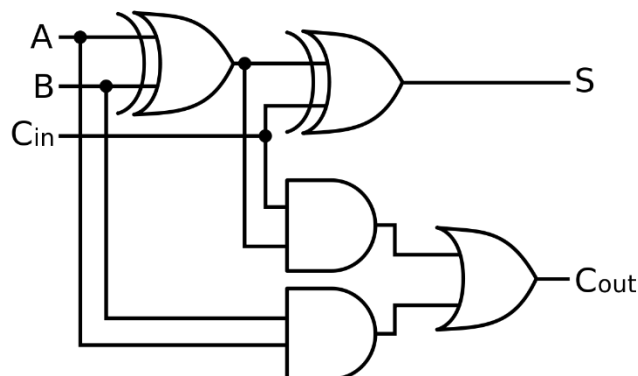


Figure 2 Logic circuit of FA.

➤ Observe the last column of the 1-bit FA truth table – the “Carry Status” column – there are three keywords you need to take care of: **delete**, **propagate**, and **generate**. These are analyzed as follows:

- Generate: concept even if $C_{in} = 0, C_{out} = 1$
 - $Generate \equiv G = AB$
- Delete: concept even if $C_{in} = 1, C_{out} = 0$
 - $Delete \equiv A'B'$
- Propagate: even does not generate if $C_{in} = 1, C_{out} = 1$
 - $Propagate \equiv P = A'B + AB' = A \oplus B$

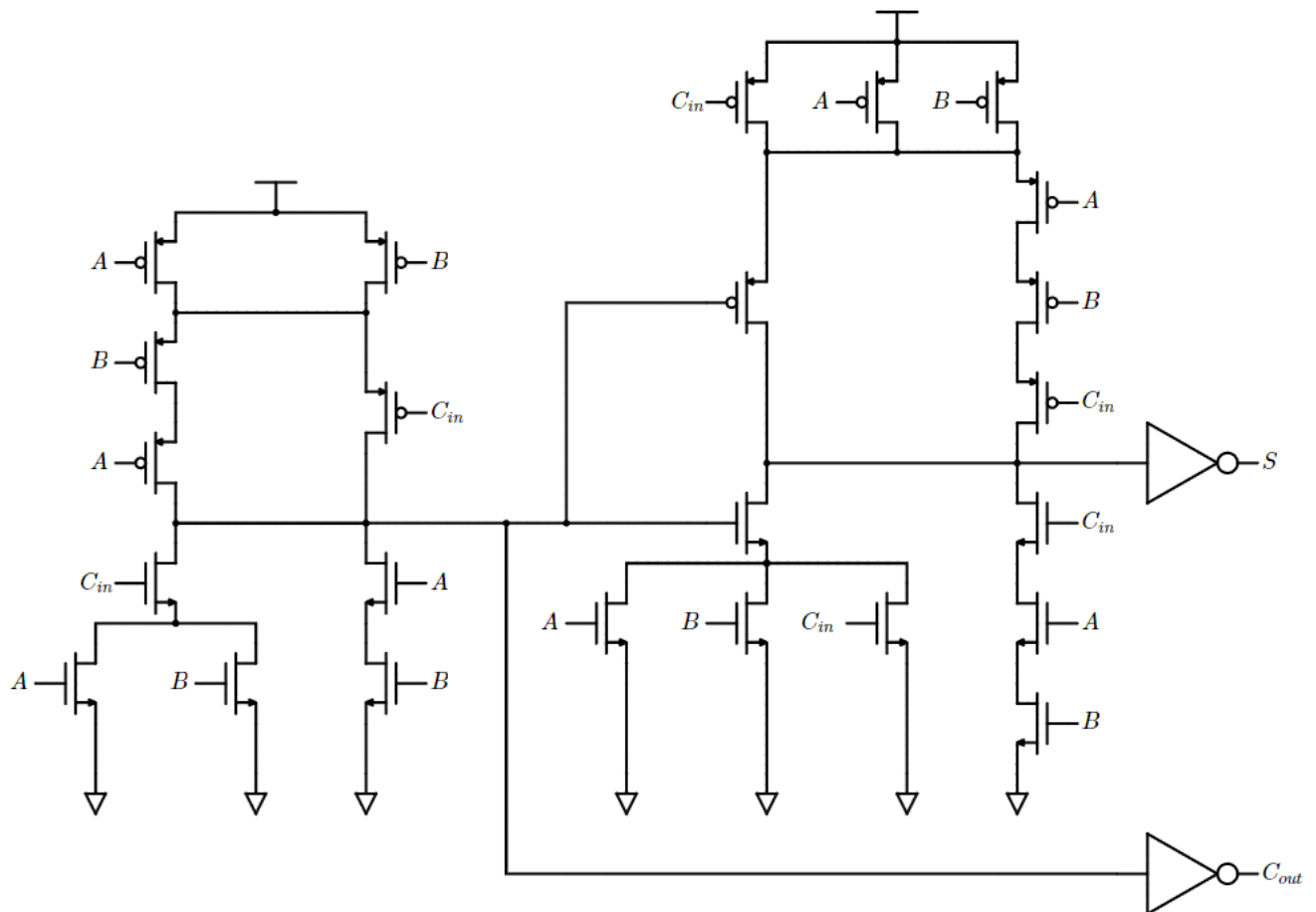
These keywords can be used to implement a Carry Look Ahead Adder to improve the critical path in the Ripple Carry Adder.

➤ In **Figure 2**, it needs 46 transistors in total to implement. This indicates that the area of this topology is large, so solutions to reduce the number of transistors are needed.

➤ Students can use the Boolean function to implement a compound gate. However, the equation describing the *sum output exists in another form*:

- $S = ABC_{in} + C_{out}'(A + B + C_{in})$
- $C_{out} = (A + B)C_{in} + AB$

According to the equation, the C_{out}' will be created at the first stage in this topology. The sum will be calculated later.

**Figure 3** 28T Full Adder.

However, this topology still has some disadvantages, such as long stacking PMOS, loading to C_{out} ($2C_D + 6C_G + C_W$)...

➤ *Mirror Adder* is the solution for this issue.

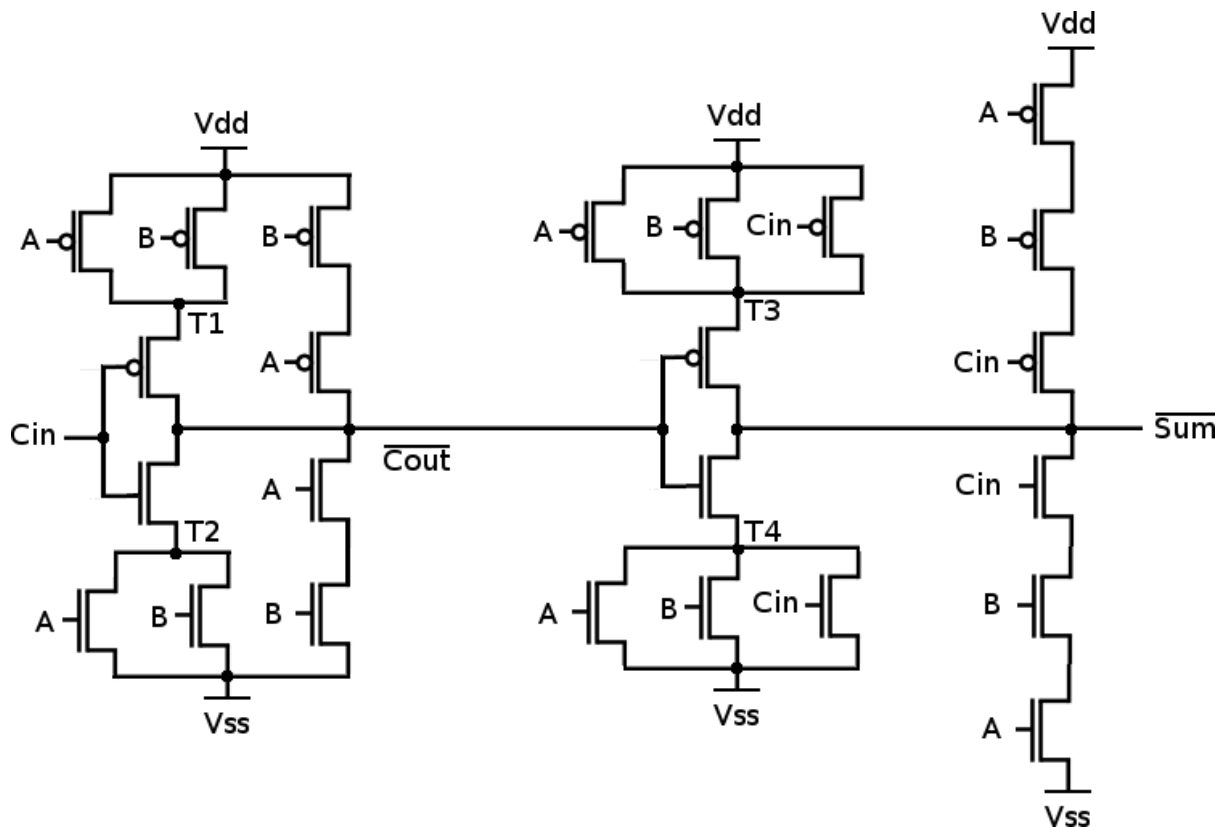


Figure 4 24T Full Adder using Mirror Adder.

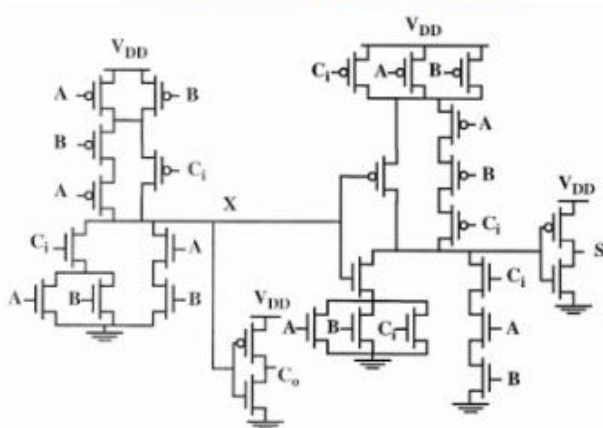
Straight forward Boolean equations

$$C_{out} = A \cdot B + B \cdot C_{in} + A \cdot C_{in}$$

$$S = A \cdot \bar{B} \cdot \bar{C}_{in} + \bar{A} \cdot B \cdot \bar{C}_{in} + \bar{A} \cdot \bar{B} \cdot C_{in} + A \cdot B \cdot C_{in}$$

$$= A \cdot B \cdot C_{in} + \bar{C}_{out} \cdot (A + B + C_{in})$$

Corresponding circuit (Problem: long p-MOS chains)



Usage of inverting properties for pull-up network

$$C_{out} = \overline{\bar{A} \cdot \bar{B} + \bar{B} \cdot \bar{C}_{in} + \bar{A} \cdot \bar{C}_{in}}$$

$$S = \bar{A} \cdot \bar{B} \cdot \bar{C}_{in} + C_{out} \cdot (\bar{A} + \bar{B} + \bar{C}_{in})$$

Improved corresponding circuit (Removal of 4 p-MOS transistors)

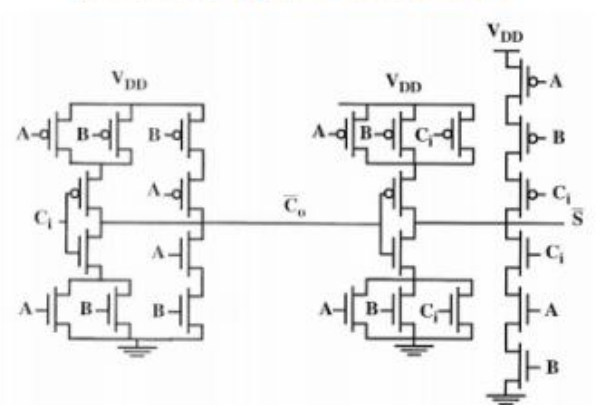


Figure 5 Summarized the topologies and equations of a 1-bit Full Adder.

Check: Students must show in their report these results

➤ The waveform to prove that the circuit in **Figures 2** and **3** works correctly. In addition, create layouts for each circuit, then show DRC confirmation and corresponding schematic with proof of LVS, where the layout of the circuit in **Figure 2** can be created by connecting the logic gates' layout from lab 2.

➤ The measurement result is shown in **Table 2**. Moreover, please present how to measure or calculate rising/falling time, propagation delay, and speed of design.

➤ In addition, define the speed of your design (hint: use the critical path).

Parameters	Original FA (Figure 2)	FA 24T (Figure 3)
t_{rise} - Rising time (10%-90%)		
t_{fall} - Falling time (90%-10%)		
t_{pd} - Average propagation delay (50% - 50%)		
Power consumption		

Table 2 Measurement results of a 1-bit Full Adder.

Question:

➤ Due to the topology shown in **Figure 3**, why do people implement the PDN of the first stage using the equation? $C_{out} = (A + B)C_{in} + AB$ instead of $C_{out} = AB + AC_{in} + BC_{in}$?

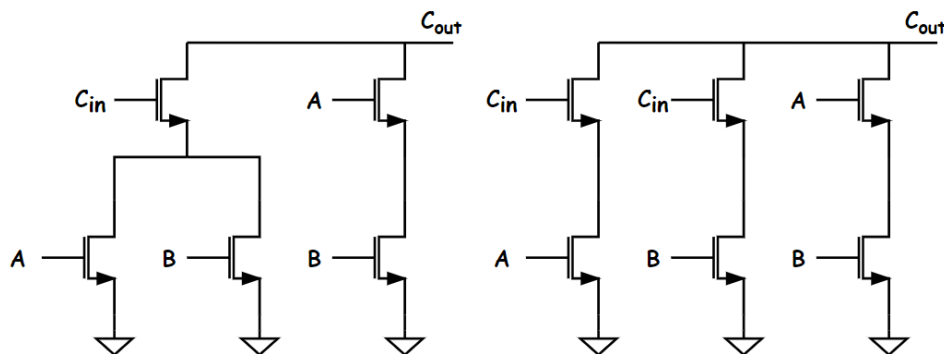


Figure 6 The PDN presents (left) $C_{out} = (A + B)C_{in} + AB$ (right) $C_{out} = AB + AC_{in} + BC_{in}$.

EXPERIMENT 2

Objective: Know how to design a basic sequential circuit.

Requirement: You will learn PRBS or pseudo-random binary sequence.

Instructions:

➤ A pseudo-random binary sequence (PRBS), which is a type of algorithm-generated random signal, is often used as model data to test high-speed serial interface devices for emulating a mission mode. It is a mathematically randomized bit stream, so that is well-neutralized and balanced data. A PRBS bit stream can be generated by using a **linear feedback shift register (LFSR)**.

- **Figure 8** illustrates an example of a 4-bit LFSR and its shifting data pattern.
- When the shift register is filled up with a seed pattern of all 1's here, the table on the right-hand side depicts how the register contents change and put out a series of PRBS.
 - Right after the final bit, it returns to the top of the bit stream.
 - There are 15 bits of pseudo-random bit stream generated.

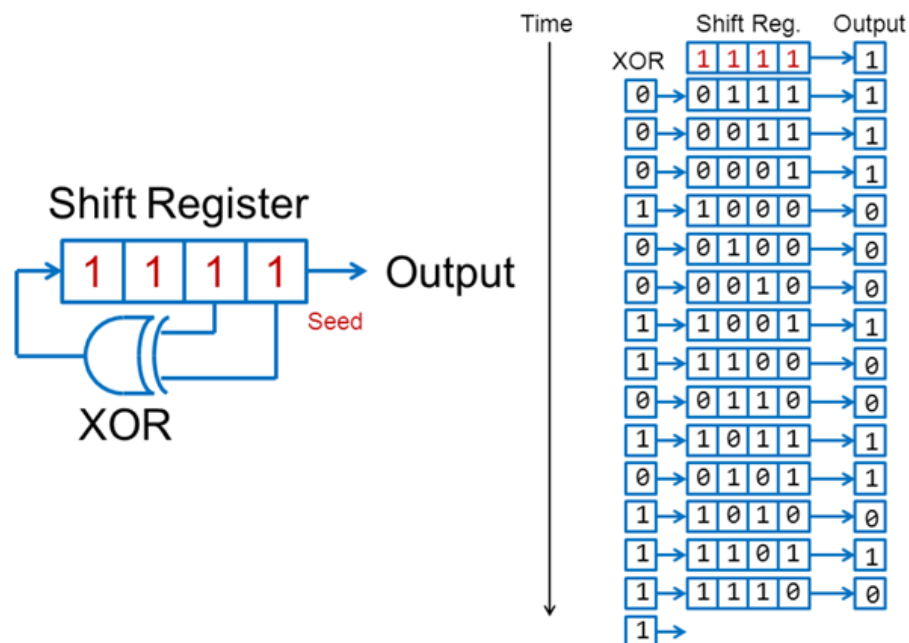


Figure 7 Linear Feedback Shift Register (LFSR).

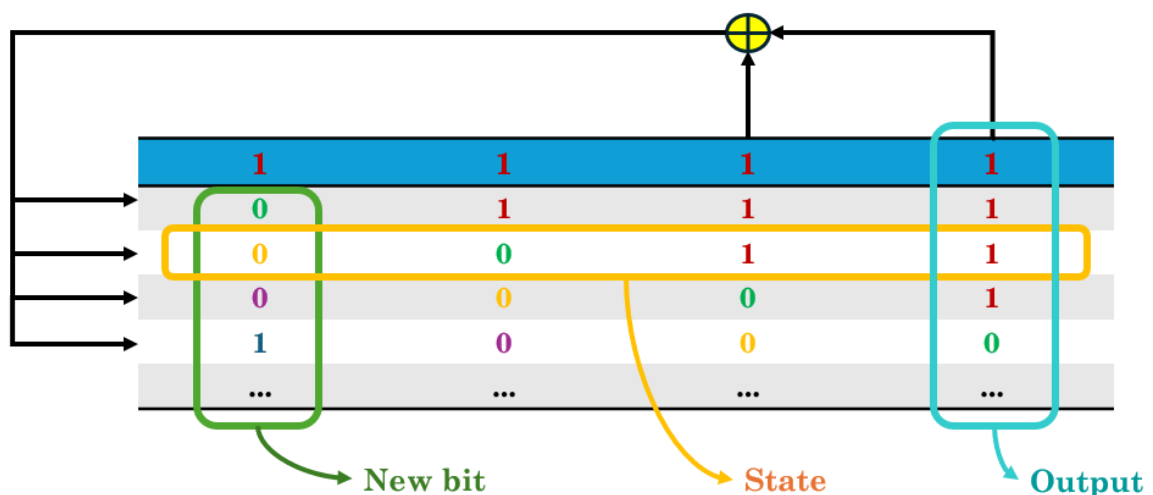


Figure 8 Illustrate the PRBS-4 algorithm in more detail.

➤ **Figure 9** illustrates how the 15-bit PRBS pattern looks like a waveform. 8 edges are toggling in the 15-bit NRZ pattern, containing eight 1's and seven 0's. This is a general feature of a PRBS shown in **Table 3**.

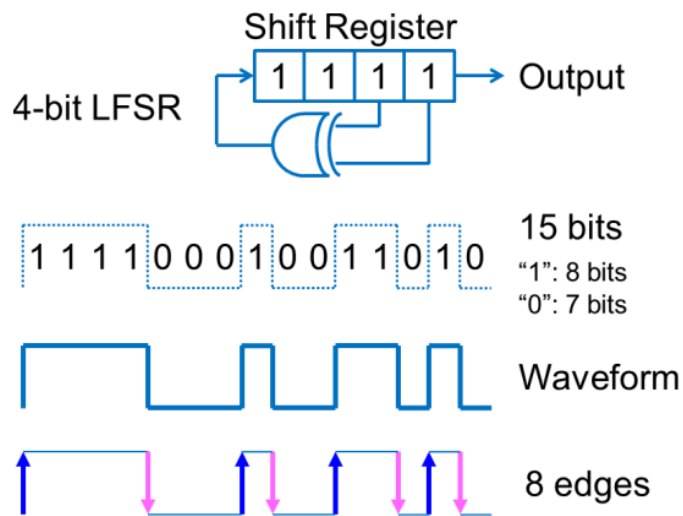


Figure 9 Waveform of PRBS by 4-bit LFSR.

LFSR n-bit	n	4	7
PRBS Length	$2^n - 1$	15	127
# of "1"s	2^{n-1}	8	64
# of "0"s	$2^{n-1} - 1$	7	63
# of edges	$2^{(n-1)}$	8	64

Table 3 Features of LFSR and PRBS.

➤ An n-bit LFSR generates $2^n - 1$ bits of PRBS. By carefully looking at the bit pattern in the shift register, you can see that all 4-bit combinations appeared except all 0's. If students feed the pattern of "0000", the shift register would be stuck, and it generates only 0's infinite. A seed pattern must not be all 0's. So, one of the 15 4-bit patterns can be accepted as a seed.

➤ The linear feedback shift register is implemented as a series of Flip-Flops as a [shift register](#). Several taps of the shift register chain are used as inputs to either an **XOR** or **XNOR** gate. The output of this gate is then used as *feedback* to the beginning of the shift register chain, hence the Feedback in LFSR.

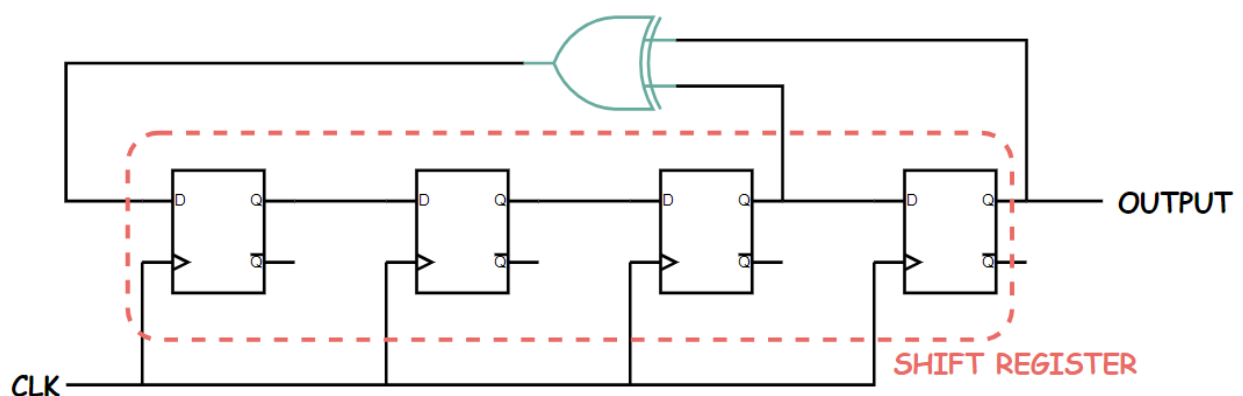
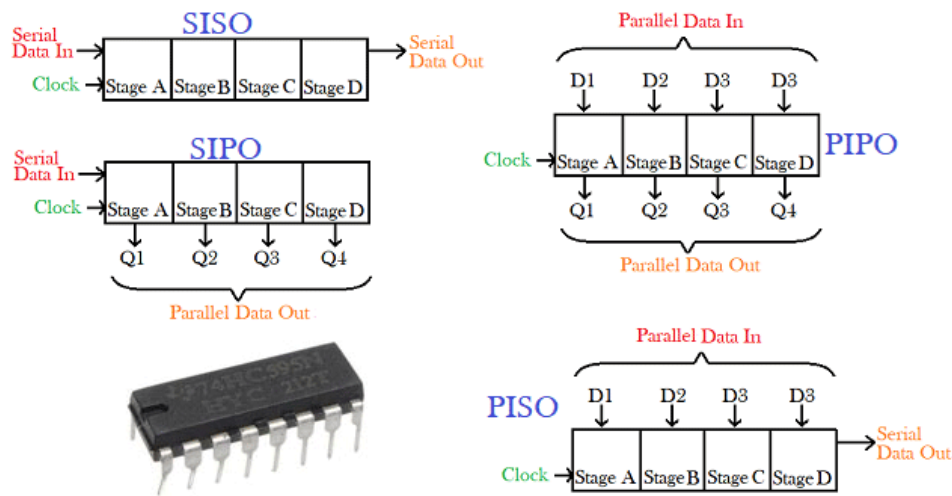


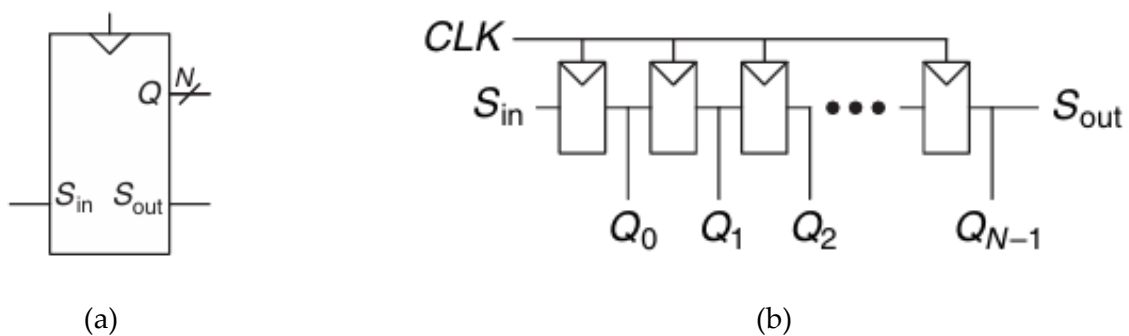
Figure 10 4-Bit LFSR using an XOR gate.

➤ Students can use any schematic of DFF they want, even the design presented in laboratory 2. In addition, students should study some types of shift registers that already exist, such as SIPO, SISO, PISO, and PIPO.

**Figure 11** Some types of shift registers.

➤ In general, a *shift register* has a clock and a serial input. S_{in} , a serial output S_{out} , and N parallel outputs $Q[N-1:0]$. In case DFF is positive edge-triggered, on each rising edge of the clock, a new bit is shifted in from S_{in} , and all the subsequent contents are shifted forward. The last bit in the shift register is available at S_{out} . Shift registers can be viewed as *serial-to-parallel converters*. The input is provided serially (one bit at a time) at S_{in} . After N cycles, the past N Inputs are available in parallel at Q .

➤ A shift register can be constructed from N flip-flops connected in series, as shown in **Figure 13(b)**. Some shift registers also have a reset signal to initialize all the flip-flops.

**Figure 12** (a) Shift register symbol (b) Shift register schematic.

➤ Before simulating your PRBS generator, please remember to transmit input data for your design. This work can be done by using a *vbit* source (in analogLib) with a MOS device to isolate when the transmitting data period is complete or set up some **initial conditions** (ADE Explorer >Simulation>Convergence Aids>Initial Condition) ... However, for any method used, please explain it.

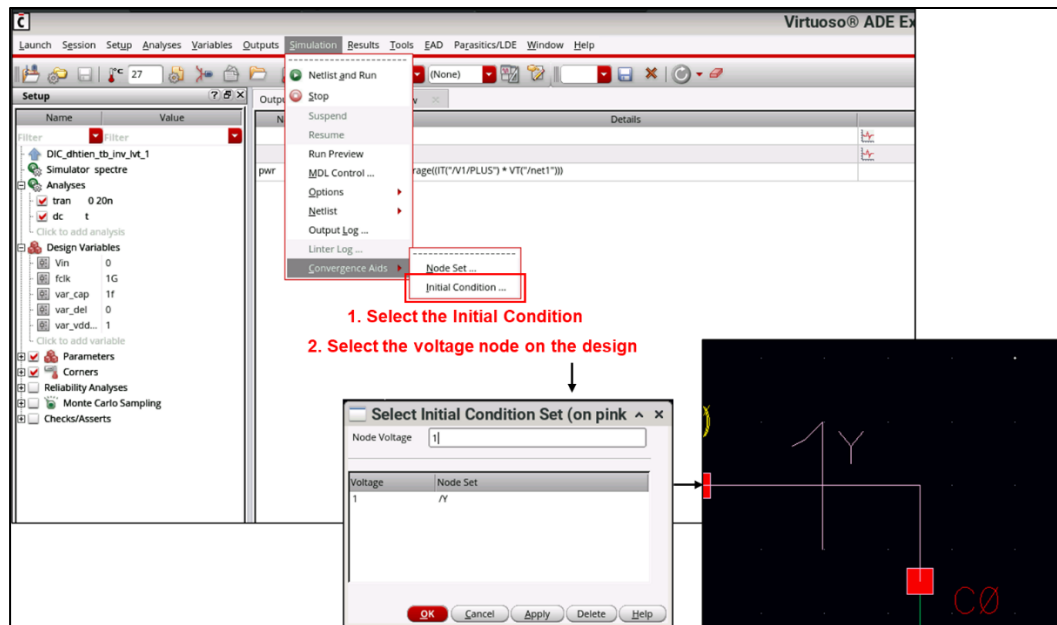


Figure 13 Select Initial Condition dialogue box.

Check:

- Show the schematics of your design, and waveform prove your design works correctly.
- Define and show how to measure clock speed.
- Students can refer to this Python code to check your results or any online generator found.

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
Created on Tue Oct 10 22:48:56 2023
```

```
@author: nelsonn
```

```
"""
```

```
import matplotlib.pyplot as plt
```

```
#Generate binary string using PRBS
```

```
#number of bits you want
n = int(input('Number of bit of PRBS you want: '))

#random a number from  $0 \leq x < 2^n - 1$ 
state = int(input('Initial state: '), 2)

output_PRBS = []

for i in range(2**n - 1):
    print(state & 1, end = " ")
    output = state & 1
    output_PRBS.append(output)
    newbit = (state ^ (state >> 1)) & 1
    state = ((state >> 1) | (newbit << (n-1)))

print('A randomized bit stream: ', output_PRBS)
print('Length of randomized bit stream = ', len(output_PRBS))

#Plotting
plt.tight_layout()
plt.figure(figsize=(15, 8))
plt.step(range(len(output_PRBS)), output_PRBS, drawstyle='steps', color='r')
plt.title('PRBS')
plt.ylabel('Voltage Level')
plt.grid(True)
plt.show()
```

Figure 14 PRBS generator in the Python programming language.

EXPERIMENT 3:

Objective: Modeling the VerilogA code for the 4-bit counter-up and 4-bit one-hot bubble code decoder. Then create a testbench with a MUX 16-to-1.

Requirement: Complete the VerilogA code for each required block, show, and verify the MUX 16-to-1.

Instructions:

In the process of designing and verifying integrated circuits, especially mixed-signal systems that combine analog and digital blocks, behavioral modeling plays a crucial role. Verilog-AMS, including Verilog-A, is a powerful hardware description language that enables flexible and efficient modeling of ideal circuits and analog–digital functional blocks.

In this laboratory, Verilog-AMS is used to construct ideal circuit models, focusing on describing the functional behavior rather than transistor-level physical details. This approach simplifies analysis, reduces simulation time, and allows designers to quickly verify concepts and system-level operation.

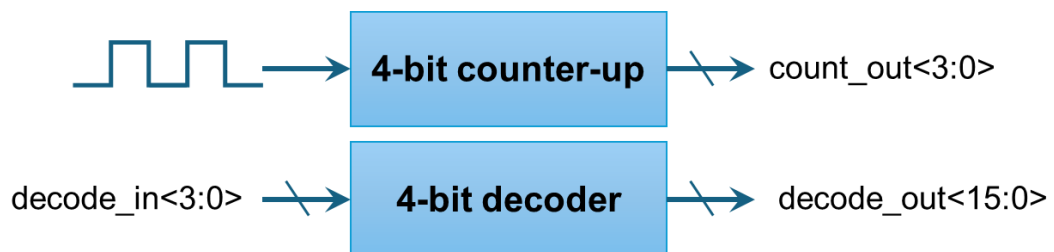


Figure 15. Two assigned blocks with in/out description.

First, for the 4-bit counter-up, the required input is simply a continuous clock at **200 MHz**; the sequence for this block is a 4-bit binary output, count-up for every **positive edge** of the input clock signal, and repeat the counting after every counting period.

Counting Period (N)	counter_out<3:0>
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
16 → repeated	0000

Table 4. The 4-bit counter-up sequence/truth table.

Second, the 4-bit one-hot bubble code is demonstrated by the truth table below:

Input (decode_in<3:0>)	Output (decode_out<15:0>) (one-hot)
0000	0000 0000 0000 000 1
0001	0000 0000 0000 00 1 0
0010	0000 0000 0000 0 1 00
0011	0000 0000 0000 1 000
0100	0000 0000 000 1 0000
0101	0000 0000 00 1 0 0000
0110	0000 0000 0 1 00 0000
0111	0000 0000 1 000 0000
1000	0000 000 1 0000 0000
1001	0000 00 1 0 0000 0000
1010	0000 0 1 00 0000 0000
1011	0000 1 000 0000 0000
1100	000 1 0000 0000 0000
1101	00 1 0 0000 0000 0000
1110	0 1 00 0000 0000 0000
1111	1 000 0000 0000 0000

Table 5. 4-bit one-hot bubble code decoder truth table.

Noted that inside the VerilogA code, you should configure these parameters of each cell. Show in the tables below.

Parameters	Default Value	Units
Input edge threshold	$VDD/2 = 1/2$	V
Propagation delay (in→out)	$\frac{1}{10 \times 200M}$	s
Rising/falling time of output	$\frac{1}{10 \times 200M}$	s
One value output	1 (equal to VDD)	V
Zero value output	0 (equal to VSS)	V

Table 6. VerilogA parameters

After these two VerilogA cells are created, you will implement a testbench including these VerilogA components. The purpose of that testbench will be to determine the function of a 16-to-1 Analog MUX (compound gate MUX topology – you can reference from LAB 2). The schematic of AnalogMUX is demonstrated in the figure below.

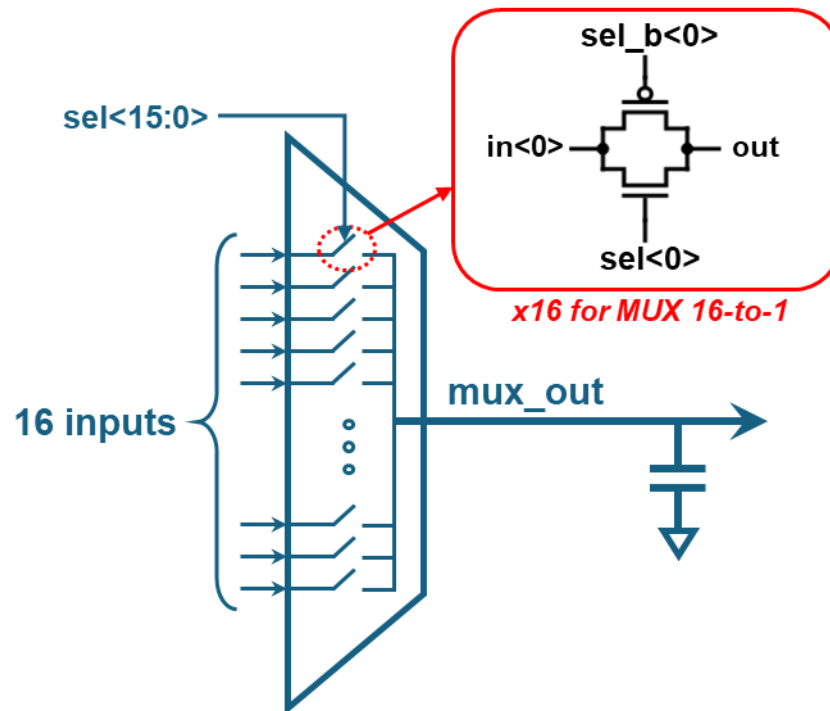


Figure 16. 16-to-1 Analog MUX

To verify this MUX and combine all the components, follow the testbench below to check the transient response for every block in that testbench.

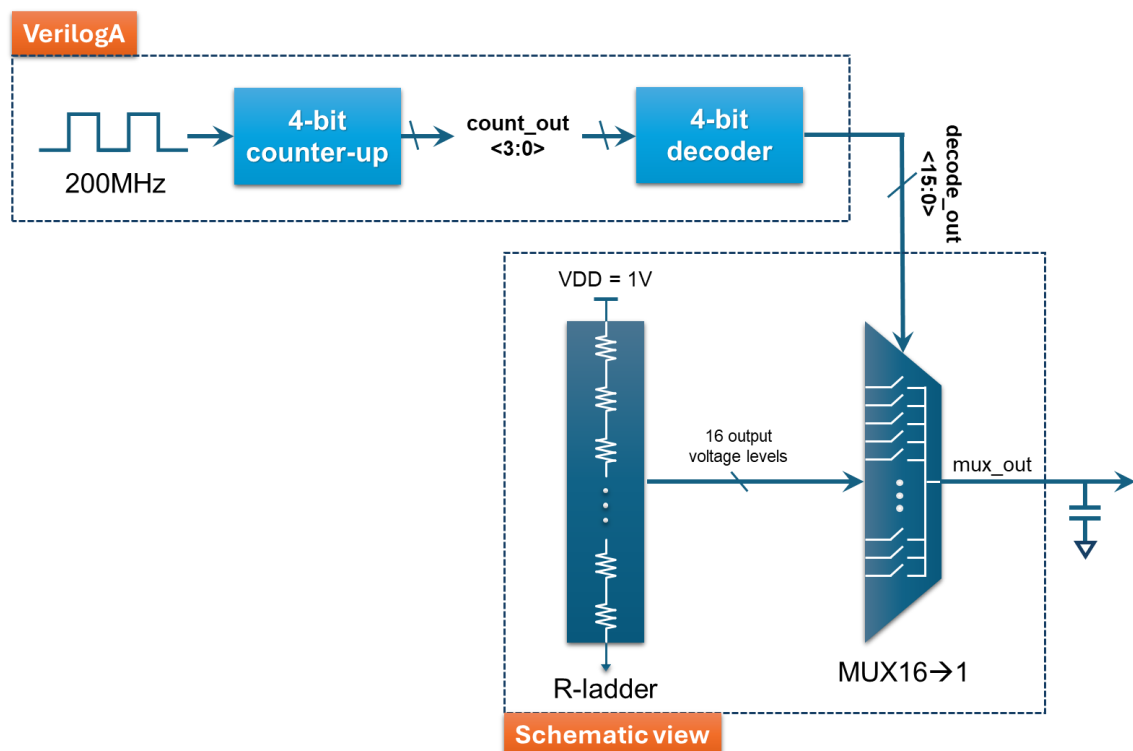


Figure 17. Full testbench for every component

In this testbench, the **200MHz** input clock goes inside the counter, the counter bits are going out from it and acting as the input for the decoder. The inputs of the MUX are represented by 16 voltage levels that come from a resistor ladder; the lowest voltage is the input<0> of the MUX (continuously increasing to the input<15>). The expected outputs are an analog signal become increase from the lowest

voltage at the first output counter and highest voltage level at the last counter output.

The table below is set up for each schematic component you have to follow:

Component	Value
Resistor	1k (Ohm)
Output capacitor	100f (F)
Input clock frequency	200M (Hz)

Table 7. Value configuration

Check:

- Show the verification results of the VerilogA blocks (transient waveform), and prove that your block is working correctly. Please attach your VerilogA code to the report.
- Implement the schematic and symbol for the Resistor ladder and the 16-to-1 MUX.
- Implement the full testbench that combines every component and show the output of the MUX that works correctly.

ACKNOWLEDGEMENTS

This laboratory has been developed and refined through the contributions of numerous individuals over the years. It was originally designed for the *EE3117 Digital IC Design* course at the University of Technology – VNU, HCM by *Mr. Nguyen Trung Hieu* (2015) and *Mr. Duong Quang Ho* (2018). Subsequent revisions and enhancements have been implemented for the *EE3117 Digital IC Design* course under the guidance of Dr. Linh Tran, with additional contributions from Doanh Bui, Tan-Khai Pham, and Phuc T. Nguyen-Phan.

Versions of this lab have been used in the following courses:

- EE3117/EE4451 Digital IC Design (2023 – present) – the University of Technology – VNU, HCM.
- EE5154 Advanced Digital IC Design (2024 – present) – the University of Technology – VNU, HCM.